

第1章 概论

Updated 2026-09-05 GMT+8

Compiled by Hongfei Yan (2026 Fall)

<https://github.com/GMyhf/dsa-modernization>

教材对应：《数据结构与算法》（张铭、王腾蛟、赵海燕，高等教育出版社 2008）第 1 章

主题与学习重点：从一个具体问题的求解入手，建立本书后面十一章共用的词汇——逻辑结构、存储结构、抽象数据类型与渐进分析。

知识点：问题抽象与建模、相邻矩阵、Floyd 最短路、最小化最大距离；
逻辑结构 (K, r) 与线性/树形/图三分；顺序、链接、索引、散列四种存储映射；
抽象数据类型 (D, R, P) 与信息隐蔽；算法的四条性质；六种算法设计方法；
渐进分析与大 $O / \Omega / \Theta$ ；最佳、最差与平均情况；时空折衷。

配套材料

类型	位置
课件 (47 页, 16:9)	DSA_CH01_Overview_ADT_Complexity.pptx
视频 (720p 轻量版)	video/ch01-preview.mp4
书稿正文	../book/ch01-adt.md
正文示例代码	../code/ch01/adt/ (modern.hpp / demo.cpp / test.cpp)
上机题参考实现	code/ch01/ (partition_count.cpp / complex_adt.cpp)

讲义里的 `cpp` 块是摘录，不是可直接编译的完整文件。 为了讲解，它们删掉了文档注释、用 ``// ...`` 略去了与本处无关的部分。
未删减的原件在上面那些代码目录里，由 ``tools/check_code.py`` 在 ``-Werror`` + `ASan/UBSan` 与 ``-O2`` 两档下真编译、真运行。
引用**原书**代码的地方一律标成 `text` —— 那些清单编译不过，所以不在 `code/` 里（与 `book/` 的 R8 同一条口径）。
`courseware/verify.py` 第 9 项会逐块核对：每个 ``cpp`` 块的有效代码行必须多数能在 `code/` 或 `courseware/code/` 里找到，凭空写的代码会让闸门变红。

本章要回答的四个问题

1. 一个现实问题怎么变成能写的程序？—— 1.1 节，股市传言。
2. 数据之间的关系怎么描述、又怎么落到内存里？—— 1.2.1、1.2.2 节。
3. 「有哪些运算」和「运算怎么实现」怎么分开？—— 1.2.3 节，抽象数据类型。
4. 两个算法谁更快，凭什么下这个结论？—— 1.3、1.4 节。

一句话概括本章：**程序 = 数据结构 + 算法** (Wirth)。算法描述求解过程，数据结构刻画信息及其关系，二者互相依赖——**结构选错了，算法再巧也补不回来**。

求解路径通常是：把现实问题抽象成模型 → 选定逻辑结构与存储方式 → 设计算法 → 写成可运行的程序。

1.1 问题求解

1.1.1 问题描述：股市的传言

若干股票经纪人之间互相传消息。谁能直接传给谁、传一次要花多久，都是已知的。

问：从哪一个人开始散布消息，能让所有人最快都知道？

注意题面不是「出边最多的人」。要的是让最后一个收到消息的人也尽可能早收到。

本章用五个经纪人 $B_1 \dots B_5$ 、10 条有向边作为实例。

方向和权都重要： $B_1 \rightarrow B_5$ 是 3，而 $B_5 \rightarrow B_1$ 是 5——

同一对人，两个方向的时间不一样。

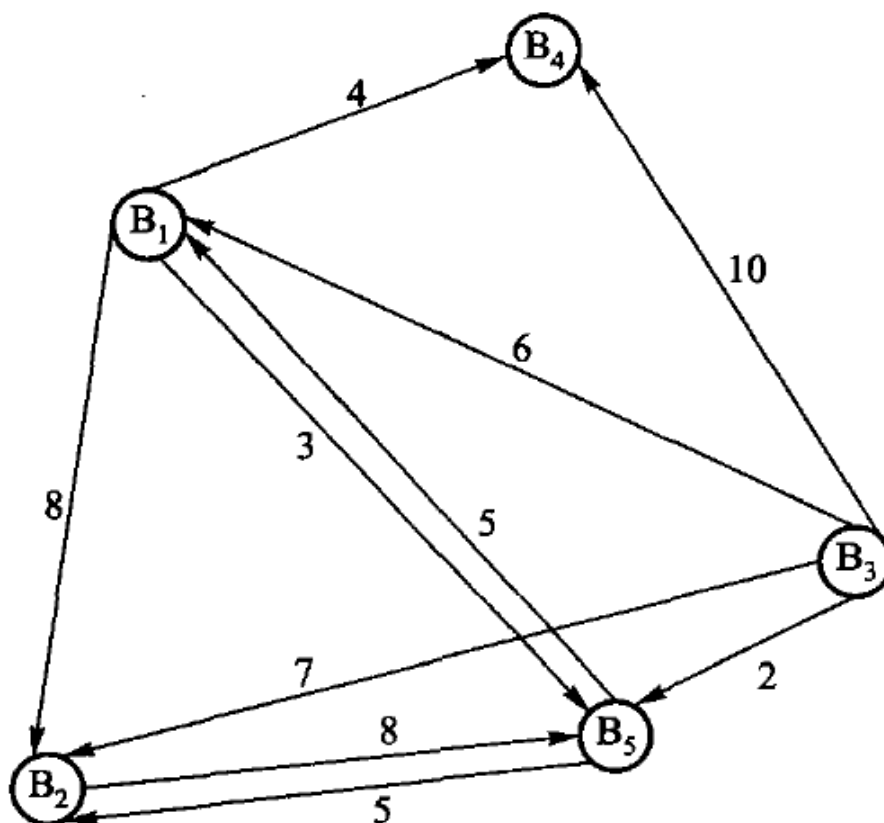


图 1.1 经纪人间的消息传递图。顶点 B_i 是第 i 个经纪人，有向边 $\langle B_i, B_j \rangle$ 表示 i 能把消息直接传给 j ，边上的数是所需时间。

起点	出边 (终点 / 耗时)
B1	$\rightarrow B_2$ (8) $\rightarrow B_4$ (4) $\rightarrow B_5$ (3)
B2	$\rightarrow B_5$ (8)
B3	$\rightarrow B_1$ (6) $\rightarrow B_2$ (7) $\rightarrow B_4$ (10) $\rightarrow B_5$ (2)
B4	(没有出边)
B5	$\rightarrow B_1$ (5) $\rightarrow B_2$ (5)

B3 指出四条边，是唯一能到达其余所有人的起点——但「能到达」还不等于「最快」。

1.1.2 问题分析和抽象

把五个人看成五个顶点，一条 $B_i \rightarrow B_j$ 边表示消息能直接传过去，边权是传播时间；没有边就记 ∞ ，表示消息永远传不到那里。

于是问题变成：在有向带权图里找一个顶点，从它出发能以最短时间把消息送到其余所有顶点。

两点之间可以绕道。 B_5 到 B_4 没有直接边，要经 B_1 中转，顶点序列 $B_5B_1B_4$ 就是一条路径，路径长度是各边权之和；两点之间最短的那条路径叫最短路径。

本题要算任意两点之间的最短路径，所以用相邻矩阵存：

第 i 行第 j 列是边 $\langle B_i, B_j \rangle$ 的权，没有边记 ∞ 。

	B1	B2	B3	B4	B5
B1	0	8	∞	4	3
B2	∞	0	∞	∞	8
B3	6	7	0	10	2
B4	∞	∞	∞	0	∞
B5	5	5	∞	∞	0

图 1.2 经纪人消息传递的相邻矩阵，这就是算法的输入。对角线是 0：一个人到自己不需要传播时间。

原书算法 1.1 的工作可以拆成三步：

1. **Floyd** 算出任意两人间的最短传播时间；
2. 对每一个候选起点，找出「从它出发到每个人的最短时间」中最慢的一项；
3. 从这些最大值中选最小的一个；某行仍有不可达的 ∞ 时，该起点不合格。

对照原书的记号： D 是代码里的 `shortest` 矩阵，`max[i]` 是每行的 `largest_distance`，`pos` 是 `best_source()` 的返回值。下标从 0 开始，因此 B3 的返回值是 2。

为什么要取「最慢的一项」

假设从 B3 开始，Floyd 算出的最短传播时间为：到 B1 是 6，到 B2 是 7，到自身是 0，到 B4 是 10，到 B5 是 2。消息要「传遍所有人」，就必须等最慢的 B4 收到，因此 B3 的完成时间是这五个数里的最大值 10——不是它们的和，也不是最小值。

把每个人都当一次起点，结果如下。 ∞ 表示至少有一人永远收不到消息，所以那一行没有可用的完成时间。

起点	到 B1	到 B2	到 B3	到 B4	到 B5	最慢的最短时间	是否可选
B1	0	8	∞	4	3	∞	否
B2	13	0	∞	17	8	∞	否
B3	6	7	0	10	2	10	是
B4	∞	∞	∞	0	∞	∞	否
B5	5	5	∞	9	0	∞	否

这张表就是图 1.3——Floyd 算完之后的结果矩阵。答案是 B3。

这个目标常称为「最小化最大距离」：不是选离某一个人最近的起点，而是选让最后一个收到消息的人也尽可能早收到消息的起点。
第 7 章讲图的时候还会再遇到它。

顺带一提：图 1.2 在 OCR 底稿里被打坏过——大量 ∞ 被认成了 8。

上面这张表的取值以图 1.1 和图 1.3 的扫描件为准，并与结果矩阵逐格核对过。

1.1.3 数据结构和算法设计

程序分成两个文件，这就是把「怎么展示」与「怎么算」分开：

- `modern.hpp` —— 只有数据和计算，不含任何输入输出；
- `demo.cpp` —— 只负责组装输入、打印结果。

用起来： `demo.cpp`

```
1  #include "modern.hpp"
2
3  #include <iostream>
4
5  int main() {
6      // B1 ... B5 对应下标 0 ... 4。
7      dsa::adt::RumorNetwork network(5);
8      network.add_route(0, 3, 4);    // B1 -> B4, 耗时 4
9      network.add_route(0, 4, 3);    // B1 -> B5, 耗时 3
10     network.add_route(0, 1, 8);    // B1 -> B2, 耗时 8
11     network.add_route(1, 4, 8);    // B2 -> B5, 耗时 8
12     network.add_route(2, 0, 6);    // B3 -> B1, 耗时 6
13     network.add_route(2, 1, 7);    // B3 -> B2, 耗时 7
14     network.add_route(2, 3, 10);   // B3 -> B4, 耗时 10
15     network.add_route(2, 4, 2);    // B3 -> B5, 耗时 2
16     network.add_route(4, 0, 5);    // B5 -> B1, 耗时 5
17     network.add_route(4, 1, 5);    // B5 -> B2, 耗时 5
18
19     const auto source = network.best_source();
20     if (source) {
21         std::cout << "最佳传播起点是 B" << *source + 1 << '\n';
22     } else {
23         std::cout << "不存在能到达全部经纪人的起点\n";
24     }
25 }
```

在仓库根目录：

```
1  c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch01/adt code/ch01/adt/demo.cpp -o /tmp/rumor-demo
2  /tmp/rumor-demo
```

第一行是「编译」，第二行是「运行刚刚编译出的程序」。命令逐项解释：

片段	含义
<code>c++</code>	C++ 编译器命令。macOS 上通常指 Clang，Linux 上也可能指 GCC
<code>-std=c++17</code>	按 C++17 语言规则编译；本书的 <code>std::optional</code> 需要它
<code>-Wall -Wextra</code>	打开常见与额外的编译器警告（未使用变量、可疑转换等）
<code>-Werror</code>	把警告当作错误；学习时可及早发现问题，初学调试可暂时去掉
<code>-Icode/ch01/adt</code>	加一个头文件搜索目录， <code>#include "modern.hpp"</code> 才找得到
<code>-o /tmp/rumor-demo</code>	指定输出的可执行文件名

输出：

```
1 | 最佳传播起点是 B3
```

改一改再跑一次是理解这段代码最快的方式：

- 删掉 B3 的一条出边（例如 `network.add_route(2, 1, 7);`），B3 就到不了 B2；此时不存在能覆盖全部顶点的起点，`best_source()` 返回空值，程序打印「不存在能到达全部经纪人的起点」。
- 把 `B3 -> B4` 的时间从 10 改成 1，答案仍是 B3，只是它的最慢传播时间从 10 缩短到 7。

模型：`RumorNetwork` 的对外接口

```
1 class RumorNetwork {
2 public:
3     // infinity 只表示「没有直接路线」；每段时间都必须小于它
4     static constexpr int infinity = std::numeric_limits<int>::max() / 4;
5     // 最短时间是若干段之和，用 64 位存；「到不了」另记 unreachable
6     using distance_type = std::int64_t;
7     static constexpr distance_type unreachable = std::numeric_limits<distance_type>::max();
8
9     explicit RumorNetwork(std::size_t people);           // 建 n x n 距离矩阵
10    void add_route(std::size_t from, std::size_t to, int cost); // 加一条有向边
11    [[nodiscard]] std::optional<std::size_t> best_source() const; // 算答案
12
13 private:
14     std::vector<std::vector<int>> distance_; // 相邻矩阵；末尾下划线 = 私有成员
15 };
```

这就是这个 ADT 的全部：三个运算 + 一份内部表示。调用者不该关心后者。

几个细节值得单独说：

- `#pragma once` 告诉编译器：同一个头文件即使被间接包含多次，也只处理一次。
- `namespace dsa::adt { ... }` 把名字放进命名空间，完整类名是 `dsa::adt::RumorNetwork`，避免和别人写的 `RumorNetwork` 重名。
- 构造函数冒号后的部分叫成员初始化列表，先构造 `distance_`（5 行 × 5 列、初值全是 `infinity`），再进入花括号把对角线改成 0。
- `add_route` 的第一段 `if` 检查编号越界与负时间，不合法就抛 `std::invalid_argument`；第二段 `if` 只在新边更短时才更新，因此重复添加同一方向的边会保留较小的那个。
- 末尾下划线 `distance_` 是本书的私有成员约定。数据成员绝不和成员函数重名——

那正是原书遇到的第一处编译错误（第3章 `int top` 撞上 `bool top(T&)`）。

Floyd：三重循环到底在循环什么

```
1  const std::size_t people = distance_.size();
2  std::vector<std::vector<distance_type>> shortest(           // 64 位：路径是若干段之和
3      people, std::vector<distance_type>(people, unreachable));
4  for (std::size_t from = 0; from < people; ++from) {
5      for (std::size_t to = 0; to < people; ++to) {
6          if (distance_[from][to] != infinity) {             // 有直接路线才抄进来
7              shortest[from][to] = distance_[from][to];
8          }
9      }
10 }
11 for (std::size_t via = 0; via < people; ++via) {           // 允许经过 via
12     for (std::size_t from = 0; from < people; ++from) {     // 固定起点
13         for (std::size_t to = 0; to < people; ++to) {       // 固定终点
14             if (shortest[from][via] != unreachable &&       // 两段都可达
15                 shortest[via][to] != unreachable &&
16                 shortest[from][to] > shortest[from][via] + shortest[via][to]) {
17                 shortest[from][to] = shortest[from][via] + shortest[via][to];
18             }
19         }
20     }
21 }
```

`infinity` 只表示「两人之间没有直接路线」；最短时间是若干段之和，可能超过 `int`，所以另开一个 64 位矩阵 `shortest`，「到不了」记为 `unreachable`。

若仍用 `int` 并借 `infinity` 表示到不了，一条总长恰好等于 `infinity` 的合法路径会被误读成到不了，更长的还会溢出（本书决策 D-039）。

三层循环的含义不是「随便循环三次」，而是按中转站逐步扩大可用路径：

```
1  for each via:      允许路径经过 via
2      for each from: 固定起点 from
3          for each to: 固定终点 to
4              比较 原来的 from->to 与 from->via->to
```

例如 B2 到 B4 没有直接边，但 B2→B5 是 8、B5→B1 是 5、B1→B4 是 4，所以 Floyd 最终得到 B2→B4 是 $8 + 5 + 4 = 17$ 。

⚠ 最外层必须是 `via`。把 `via` 挪到最内层，算法就不再正确——

它依赖的是「前 k 个中转点都已考虑完」这个不变量。这是初学者最常犯的错。

选答案：每行取最大，再在其中取最小

```
1  std::optional<std::size_t> result;           // 空 = 还没找到合格的起点
2  distance_type smallest_eccentricity = unreachable;
3  for (std::size_t from = 0; from < people; ++from) {
4      distance_type largest_distance = 0;
5      for (distance_type distance : shortest[from]) { // 这一行的最大值 = 该起点的完成时间
6          largest_distance = distance > largest_distance ? distance : largest_distance;
7      }
8      if (largest_distance < smallest_eccentricity) {
9          smallest_eccentricity = largest_distance;
10         result = from;
11     }
12 }
13 return result;                               // 全都不可达时返回 std::nullopt
```

这里的 **eccentricity**（离心率）就是某起点到全部顶点的最短距离中的最大值。

任何一行含 `unreachable` 时，该行最大值也是 `unreachable`，不会优于有限答案；
若所有行都含 `unreachable`，`result` 保持为空，函数返回 `std::nullopt`。

时间复杂度 $O(V^3)$ ，空间复杂度 $O(V^2)$ 。这适合顶点数少、又需要比较所有起点的场合；
大图应该根据稀疏度和查询需求换别的最短路径算法（第 7 章）。

「可能没有答案」怎么返回

`best_source()` 返回 `std::optional<std::size_t>`——可以先理解成一个「可能装着下标的盒子」：有解时盒子里是起点下标，无解时盒子是空的（`std::nullopt`）。

	原书 / 传统写法	本书写法（风格公约 D-001）
签名	<code>bool find(size_t& out);</code>	<code>optional<size_t> best_source();</code>
结果怎么带出来	答案和「有没有答案」走两条通道	一个值，取之前必须先判空
忘了检查会怎样	读到未初始化的 <code>out</code>	编译期就拦住（还有 <code>[[nodiscard]]</code> ）
出错时	容器里直接 <code>cout</code> 打中文提示	容器内零 I/O，真错误抛标准异常
可测试性	输出焊死在算法里，没法自动测	纯函数，断言直接比对返回值

对照写法见第 2.2.2 节。

1.2 数据结构

数据结构描述的是：按一定逻辑关系组织起来的数据，它们在计算机里怎么存放，以及定义在其上的运算。三件事要分开看——逻辑结构、存储结构、运算——后面各章都在这三层之间做选择。

1.2.1 数据的逻辑结构

逻辑结构是从具体问题里抽出来的模型，只谈「有哪些结点、结点之间是什么关系」，不谈它们占几个字节。1.1 节经纪人之间的传播路径就是一个有向图。

从集合论的观点看，数据的逻辑结构可以用一个二元组 $B = (K, R)$ 表示：

- K 是数据结点（node）组成的有穷集合，每个结点代表一个数据或一组有明确结构的数据；
- R 是定义在 K 上的一组二元关系。本书里 R 一般只含一个关系 r ，此时记作 (K, r) 。

若 $\langle k, k' \rangle \in r$ ，则称 k 为 k' 的前驱， k' 为 k 的后继。
没有前驱的结点叫**开始结点**，没有后继的结点叫**终止结点**。

例：教学计划。 每门必修课是一个结点，全部必修课组成 K ；「必须先修」是 K 上的一个关系。
〈程序设计, 数据结构〉表示必须先上程序设计。
没有先修约束的课可以并行开设。

结点的类型。 结点是数据结构中数据的基本单位，其类型既可以是基本数据类型（整数、实数、布尔、字符、指针），也可以是复合数据类型（数组、结构、对象），而复合类型本身又能参与定义更复杂的结点类型。数据结构课关心的是结点之间的关系，所以**通常把结点当作黑盒**。

结构的分类，以关系 r 的性质为准：

结构	前驱	后继	本书在哪几章
线性结构	至多 1 个	至多 1 个	线性表、栈、队列、串（第 2-4 章）
树形结构	根无前驱，其余恰好 1 个	不限	二叉树、树、B 树（第 5、6、11 章）
图结构	不限	不限	最短路、最小生成树（第 7 章）

- 线性结构中存在唯一的开始结点和唯一的终止结点，其余都是既有唯一前驱、又有唯一后继的内部结点。
- 树形结构中有且仅有一个结点 k_0 （**树根**）没有前驱；从根到任一结点 k 存在唯一的结点序列 $k_0, k_1, \dots, k_s = k$ ，称为一条**路径**。UNIX / DOS 的文件系统就是典型的树。
- **树 vs 图**的分界：每个结点是否仅从属于一个直接前驱。
线性 vs 树的分界：每个结点是否仅有一个直接后继。

1.2.2 数据的存储结构

存储结构回答的是：同一套逻辑关系，在存储器里怎么落下来。
主存按字节编址，相邻单元地址连续，**可以按地址随机访问**，访问不同地址的时间基本相同。

对逻辑结构 (K, r) 而言，存储结构就是建立一种从逻辑结构到物理存储空间的**映射**：
一方面把每个结点 $k \in K$ 映射到一块唯一的连续存储区域，
另一方面把每个关系元组 $\langle k_i, k_j \rangle \in r$ 映射为存储单元地址之间的关系（顺序关系，或指针的地址指向关系）。

同一种逻辑结构可以采用不同的映射。 线性结构既可以顺序存储成顺序表，也可以链接存储成链表。常用的四种基本方法如下。

1. 顺序方法。 把一组结点存放在一片地址相邻的存储单元中，
结点间的逻辑关系用存储单元的自然顺序表达。按下标访问因此是 $O(1)$ ；
程序设计语言里的数组就是它的实例。

顺序存储结构通常也称**紧凑存储结构**：存储空间除数据本身外没有附加信息。
紧凑性用**存储密度**度量——数据本身占用的空间与整个结构（含附加信息）占用空间之比。
存储密度太小的结构，空间效率就低。

部分非线性结构也能顺序存储，前提是同时存一些附加信息来表示逻辑关系：
完全二叉树按层编号后，孩子下标可以用 $2i + 1$ 、 $2i + 2$ 算出来（第 5 章）。

2. 链接方法。 每个结点里附一个（或多个）指针域，专门存放后继的地址；
结点因此分成数据域和指针域。适合经常增删、长度事先不知道的结构。
代价是：不知道指针时只能从链头一个一个比下去，按位置访问不再是 $O(1)$ 。

3. 索引方法。顺序存储的一种推广：造一个从整数域 Z 到存储地址域 D 的索引函数 $Y: Z \rightarrow D$ ，形成一张存了一组指针的索引表。
索引表的存储空间是**附加在结点存储空间之外的**。

索引的主要作用是**提高检索效率**：数据量很大时检索会涉及大量磁盘读写，先用索引确定存储地址再读写，可以大幅降低读写的数据量（第 11 章）。

原书特意指出，索引函数一般**不是**数组那样的简单线性函数——数据结点长度不等时它就不可能是线性的。
图 1.4 画的正是这种情况：

1	存储区的数据 —— 物理无序，而且每块长度不等				
2	+-----+-----+-----+-----+-----+				
3	[1] 73	[2] 52	[3] 42	[4] 98	[5] 37
4	+-----+-----+-----+-----+-----+				
5					
6	线性索引 —— 附加在数据之外的一张表				
7	+-----+-----+-----+-----+-----+				
8	37	42	52	73	98 关键码：升序 -> 逻辑有序
9	+-----+-----+-----+-----+-----+				
10	[5]	[3]	[2]	[1]	[4] 指向第几块：乱序 -> 物理没搬过
11	+-----+-----+-----+-----+-----+				

这两行的错位就是索引的全部意义。

- 上面一行升序，所以可以在索引里二分；
- 下面一行完全乱序，说明**数据一块都没有搬动过**；
- 要找关键码 52，先在索引里二分定位到第 3 列，读出 [2]，再直接去存储区取第 2 块——只读了一块，没有扫过其余四块。

再看每块的宽度：五块长度各不相同。正因为长度不等，「第 k 块在哪」
就没法用「起始地址 + $k \times$ 块长」算出来，只能查表——
这才是原书说「索引函数并非数组那样的简单线性函数」的意思。

代价是索引表本身的空间。这正是 1.4.3 节要讲的**时空折衷**。

4. 散列方法。索引法的延伸：不查表，而是用**散列函数**把关键码的值直接算成存储地址。
散列函数应尽可能把地址均匀分布在散列表的地址空间上，同时计算要简单以便提速。
冲突怎么处理、装载因子多高会退化，是第 10 章的内容。

实际应用中往往是几种方法的组合。例如树形结构的「子结点表」表示法就是顺序 + 链接（第 6 章）。
选存储方法时还要综合考虑定义在其上的运算：**要不要随机访问、会不会频繁插入、数据在内存还是外存**。

1.2.3 抽象数据类型

抽象意味着同一个概念可以有多种实现。一旦抽象问题被解决，许多同类问题就不必从头再来。

抽象数据类型（abstract data type, ADT）这个词是逐步形成的。
20 世纪 60 年代后期，为了把算法细节和数据的内部结构藏起来，Simula 67 引入了类，
随后出现了**模块**：模块式给出外部可见的运算接口，模块体存放对外不可见的私有数据和实现。
接口与实现分离之后，用户才能真正自定义类型，达到**数据抽象和信息隐蔽**。

ADT 可以看成「**定义了一组操作的抽象模型**」：数据和操作封在一起，
使用方只能通过这些操作访问数据，不能直接去翻内部表示。
这样，讨论一种结构时可以先不管它是数组还是链表，只问：
它提供哪些运算？这些运算有什么约定？

整数连同加、减、乘、除，就是一个最简单的 ADT。

一般写成三元组 (D, R, P) :

```
1 ADT 名字 {
2     数据对象 D: 结点是什么
3     数据关系 R: 结点之间有什么关系
4     基本操作 P: 对外提供哪些运算
5 }
```

D 和 R 给出数据模型，是对**数据**的抽象； P 给出这个模型能做什么，是对**操作**的抽象。

C++ 用类（以及类模板）的**声明**表示 ADT，用类的**定义**去实现它。

原书【代码1.2】用下面这个空模板提醒读者「先写接口」：

```
1 template <class Type>
2 class ClassName {
3 private:
4     Type data;           // 取值类型和存储 — 实现细节，外部看不到
5 public:
6     void method();       // 对外运算 — 这一行才是 ADT 的内容
7 };
```

这两层不要混：概念层上的 ADT 描述「问题需要什么运算」；
实现层上的类描述「这些运算在内存里怎么做」。所以 ADT 也可以说成：
数据的逻辑结构，加上定义在这个逻辑结构上的一组抽象运算。

本书的一处改动：不写抽象基类

原书习惯先写一个只有函数声明、没有任何实现的抽象基类（例如 `List<T>`），
再让 `arrList` 继承它。本书各章不这么做，原因有两条：

1. **它给不了多态**。基类之所以有用，是因为可以「拿基类指针指向派生对象」，
写一份代码同时适用于顺序表和链表。但本书各章的容器都是**模板**，
元素类型在编译期就定了，彼此之间根本不需要互相替换；这个基类从头到尾没有一处用到。
2. **它还会引入一个陷阱**。只要基类的析构函数不是 `virtual` 的，一旦有人写出

```
1 List<int>* p = new ArrayList<int>;
2 delete p;                               // 未定义行为
```

标准规定这是**未定义行为**——编译器不保证会发生什么：常见后果是只析构了基类那一层，
派生类里 `new` 出来的缓冲区没人释放，内存就漏了；也可能直接崩。

「未定义行为」在本书里反复出现，它的意思始终是「标准不再对程序的行为作任何承诺」，
而不是「结果不确定但还能用」。

所以本书把运算直接写在实现类上。对元素类型的要求，正文里用一句话说清即可：

例如顺序栈、顺序表底层是 `new T[n]`，会把整块槽位都默认构造出来，所以 `T` 必须能默认构造。

1.3 算法

算法的中文名称出自《周髀算经》，英文名 algorithm 源于 9 世纪波斯数学家 al-Khwarizmi。

算法的第一次编写是 Ada Byron 于 1842 年为巴贝奇分析机写的求解伯努利方程的程序，

因此她被大多数人认为是世界上第一位程序员（巴贝奇未能完成分析机，这个算法最终没在机器上跑过）。

20 世纪图灵提出图灵机模型，**解决了算法定义的难题**。

从应用范围看，算法分数值算法与非数值算法；从工作方式看，分串行算法与并行算法。本书后面各章几乎都是非数值的、串行的算法。

1.3.1 算法的概念

算法是对特定问题求解过程的描述，是指令的有限序列。
程序是算法的一种实现，计算机按照程序逐步执行算法，实现对问题的求解。

求解一个问题，通常用该问题的输入数据类型和其输出所应遵循的性质来描述。
以求最大公因子为例：输入是任意给定的两个正整数 n 、 m ，
输出是既能整除 n 又能整除 m 的所有公因子中最大的非负整数。

算法一般具有以下四条性质：

- 1. **通用性**。对那些符合输入数据类型的任意输入数据，都能求解并保证结果正确。
不是只对书上那一组样例成立。
- 2. **有效性**。由有限条能够被人或机器确切执行的指令组成，指令类型明确、
每一步的执行结果具有确定的、可预期的数据类型。
例如辗转相除法的基本动作只有整数相除求余数与商、判断余数是否为 0 这么几条。
- 3. **确定性**。每执行一步之后，关于下一步怎么执行应该有明确的指示——
可以是条件判断、分支、顺序执行下一条，或者指示算法结束。
不能缺乏下一步指令（「被锁住」），也不能含有模糊不清的指令。
- 4. **有穷性**。执行必须在有限步内结束，不能含死循环。

⚠ 注意第 4 条的一个常见误解：算法由有限条指令组成这一事实本身
并不能保证执行的有穷性——循环条件写错，有限条指令也可以转个没完。
设计算法时应该专门盯住结束条件。

1.3.2 算法设计

算法设计与算法分析是计算机科学的核心问题。常用的算法设计方法如下。

方法	要点	本书在哪儿用
穷举法 (enumeration)	把问题空间里的求解对象——列举、逐一分析处理并验证。要求对象有限、有列举规则	<code>switch</code> 就是一种穷举
回溯法 (backtrack)	按某种顺序逐一枚举并检验候选解。当前候选解不可能是解时回溯到上一步；还不够规模但其余条件都满足时向前试探	深度优先搜索（第 5–7 章）、 背包问题（第 3 章）
分治法 (divide and conquer)	把难以直接解决的大问题分割成规模较小的子问题，各个击破，再合并出整个问题的解。自顶向下	快速排序、归并排序（第 8 章），二分检索（第 10 章）
递归法 (recursion)	分治产生的子问题往往是原问题的较小模式，反复分治自然导致递归。分治与递归如同一对孪生兄弟	同上
贪心法 (greedy)	从初始状态出发，依据某种贪心标准做若干次贪心选择。只考虑局部最优，寄望于拼出全局最优	Prim、Kruskal、Dijkstra（第 7 章）
动态规划 (dynamic programming)	也把问题分解成子问题，但子问题不相互独立。用一张表记录已解子问题的答案，避免大量重复计算	Floyd（第 7 章）、最佳二叉搜索树（第 12 章）

- 全局穷举往往很费时间，但在问题的局部采用穷举常常很有效。
- 使用贪心法的核心问题是：选择一个真能产生问题最优解的最优度量标准。

- 动态规划与分治的区别就在于子问题是否独立：若子问题重叠而仍用分治，同一个子问题会被重复计算很多次。

运用算法的常见方式有四种：算法的组合；经典算法的变形与推广；研究困难问题的特殊情况；探索新的算法。

回到 1.1 节的传言问题：它要的是任意两点之间的最短路径，并且顶点数很小，所以选 Floyd，而不是对每个起点各跑一遍 Dijkstra。

1.4 算法分析

同一类问题往往存在多种算法，各有优缺点和适用场景。如何取舍，就要靠算法分析。

为比较不同算法的效率，Juris Hartmanis 和 Richard E. Stearns 提出了**计算复杂性**（computational complexity）来度量算法的难度：运行算法所需的计算机资源越多，复杂性越高。资源中最重要的是时间（处理器）和空间（存储器），所以复杂性通常体现为**时间复杂性**和**空间复杂性**两个指标。

为什么不能拿秒来衡量算法

- 同一个算法，Cray 巨型机上跑和 PC 上跑差很多；反过来，效率较差的算法在 Cray 上也可能比效率极好的算法在 PC 上更快。
- 同一台机器上，编译型语言往往比解释型快很多——原书举例：用 C 写的程序比用 LISP 写的、采用同样算法的程序快将近 20 倍。

所以评估算法的效率不能采用微秒、纳秒这样的真实时间单位。最重要的不是具体运行时间，而是**算法与输入规模之间的关系**：用一定规模（size）的数据作为输入时，程序运行所需的**基本操作**（basic operation）数。

- **规模**一般指输入量的数目。排序问题里就是待排序元素的个数。
- 完成一个**基本操作**所需的时间应该与**被操作的具体数值无关**，例如两个整数相加。

于是： $t = cn$ 时，规模增加到 5 倍，时间也增加到 5 倍； $t = \log_2 n$ 时，规模加倍只意味着时间增加一个单位。

1.4.1 渐进分析方法

表示数据规模和时间关系的函数通常相当复杂。计算时只考虑大的数据，**那些不能显著改变函数量级的部分都可以忽略**，得到的近似值在规模很大时足够接近原值。这种方法称为**渐进算法分析**（asymptotic algorithm analysis），简称渐进分析。

例如 $f(n) = n^2 + 100n + \log_{10} n + 1000$ ：

n	n^2	$100n$	$\log_{10} n + 1000$	谁说了算
1	1	100	1000	常数项贡献最大
10	100	1000	1001	$100n$ 与 常数项持平
100	10 000	10 000	1002	前两项持平
1000	1 000 000	100 000	1003	n^2 一家独大

n 大于 100 之后，第 2 项的贡献就小于第 1 项，而且 n 越大越微不足道。

⚠️ 并非任何情况下都可以忽略常数部分。问题规模很小时，各项系数和常数项会起到举足轻重的作用——用于上万个数的排序算法，也许并不适用于仅对 10 个数排序。

大 O 表示法

由 Paul Bachmann 于 1894 年引入。设 f 、 g 为从自然数到非负实数集的两个函数。

定义 1 如果存在正数 c 和 N ，使得对任意的 $n \geq N$ 都有 $f(n) \leq c g(n)$ ，则称 $f(n)$ 在集合 $O(g(n))$ 中，或简称 $f(n)$ 是 $O(g(n))$ 的。

它说的是**上限**（upper bound）： f 的增长最终至多趋同于 g 的增长。

换言之， $O(g(n))$ 只表明该算法最多会差到何种程度。

一个函数增长率的上限可能不止一个：在 $O(n)$ 中的函数也一定在 $O(n^2)$ 、 $O(n^3)$ 中。

大 O 表示法给出的是所有上限中最小的那个上限。

常用性质（ a 是不依赖于 n 的常数）：

1. f 是 $O(g)$ 、 g 是 $O(h) \Rightarrow f$ 是 $O(h)$ （传递）；
2. f 是 $O(h)$ 、 g 是 $O(h) \Rightarrow f + g$ 是 $O(h)$ （相加）；
3. an^k 是 $O(n^k)$ ；
4. $f(n) = c g(n) \Rightarrow f$ 是 $O(g)$ ；
5. 对任何正数 a 、 b ($b \neq 1$)， $\log_a n$ 是 $O(\log_b n)$ ——
任何对数函数无论底数为何，都具有相同的增长率；
6. 本书把 $\log_2 n$ 简写为 $\log n$ 。

常见的上限 $g(n)$ ：

$g(n)$	名称	$n = 16$	$n = 256$	长这样的算法
1	常数函数	1	1	不依赖于数据规模
$\log n$	对数函数	4	8	二分
n	线性增长	16	256	扫一遍
$n \log n$	阶数介于一、二阶之间	64	2048	快排 / 归并，一般出现在树形结构的算法中
n^2	二阶增长	256	65 536	双重循环
a^n	指数增长	65 536	—	穷举子集，往往出现在递归定义的函数计算中

指数增长比任何高次的多项式函数（ n^3 、 n^4 ）都增长得更快。

令时间单位为 $1 \mu\text{s}$ ，在 $n = 1000$ 时：

增长率	时间开销
$T(n) = n$	$1000 \mu\text{s} = 1 \text{ ms}$
$T(n) = n^2$	$10^6 \mu\text{s} = 1 \text{ s}$
$T(n) = n^3$	1000 s （约 16 分钟）
$T(n) = 2^n$	约 10^{286} 年

迄今地球的年龄还不超过 10^{10} 年。具有指数增长率的算法简称指数爆炸型算法，应用时需要谨慎。

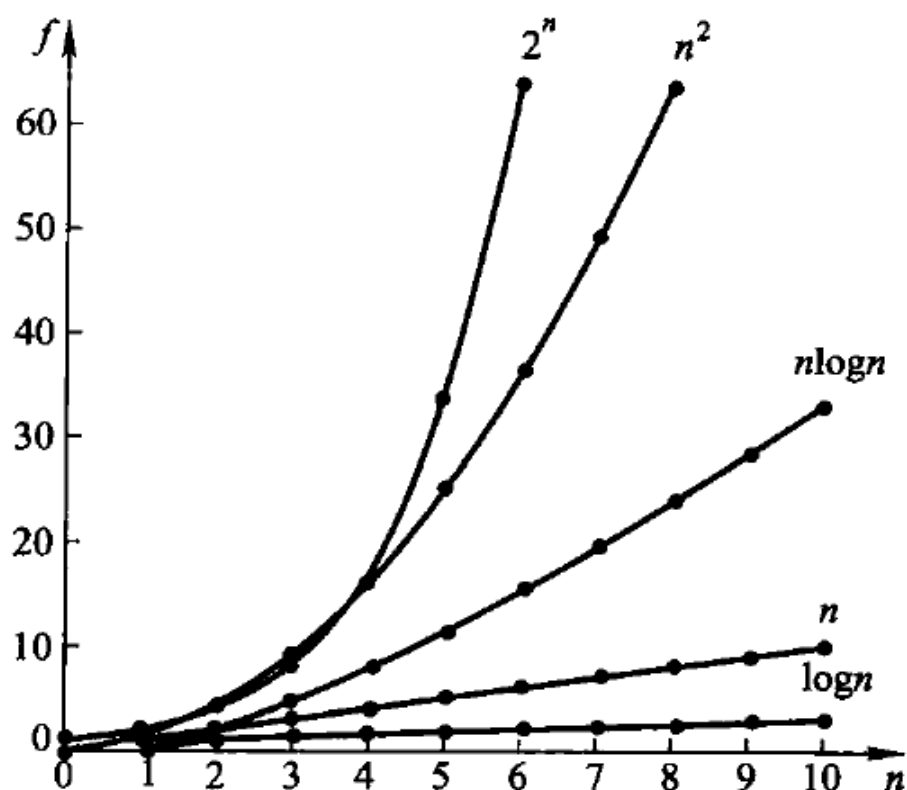


图 1.5 常用函数的增长趋势。

Ω 表示法与 Θ 表示法

定义 2 如果存在正数 c 和 N , 使得对所有的 $n \geq N$ 都有 $f(n) \geq c g(n)$, 则称 $f(n)$ 在集合 $\Omega(g(n))$ 中。

$c g(n)$ 是 $f(n)$ 取值的下限 (lower bound): f 的增长最终至少趋同于 g 。

大 O 与 Ω 的唯一区别是不等式的方向;

正如大 O 取的是所有上限中最小的那个, Ω 取的是所有下限中最大的那个。

Θ 表示法 用于上、下限同阶的情形: 若 f 既在 $O(g(n))$ 中又在 $\Omega(g(n))$ 中, 则称其为 $\Theta(g(n))$, 即存在正常数 c_1 、 c_2 与正整数 N , 使 $n > N$ 时

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

例: $f(n) = 100n^2 + 5n + 500$, 取 $c_1 = 100$ 、 $c_2 = 105$ 、 $N = 10$, 于是 $f(n)$ 是 $\Theta(n^2)$ 。这比「 f 是 $O(n^2)$ 」是更强的一句话。

渐进分析的实例：数一数赋值操作

(一) 单层循环 —— $O(n)$

```
1 for (i = sum = 0; i < n; i++)    // 循环开始前 2 次赋值: i、sum
2     sum += a[i];                // 每轮 2 次赋值: sum、i
```

循环开始前有 2 次赋值; 循环进行 n 次, 每次 2 次赋值。

总共 $2 + 2n$ 次, 渐进复杂度 $O(n)$ 。

(二) 嵌套循环 —— $O(n^2)$

```

1  for (i = 0; i < n; i++) {                                // 循环前 1 次: i
2      for (j = 1, sum = a[0]; j <= i; j++)                  // 每轮外层 3 次: i、j、sum
3          sum += a[j];                                       // 内层每轮 2 次, 共执行 i 次
4      cout << "sum for subarray 0 through " << i << " is " << sum << endl;
5  }

```

$$1 + 3n + \sum_{i=1}^{n-1} 2i = 1 + 3n + 2(1 + 2 + \cdots + n - 1) = 1 + 3n + n(n - 1) = O(n^2)$$

(三) 嵌套 $\neq$ 平方 —— 仍是 $O(n)$

```

1  for (i = 4; i < n; i++)                                // 外层 n - 4 次
2      for (j = i - 3, sum = a[i - 4]; j <= i; j++)        // 内层恒定 4 次, 与 i 无关
3          sum += a[j];                                     // 每个 i 合计 8 次赋值

```

外层进行 $n - 4$ 次; 对每个 i , 内层只执行 4 次, 次数与 i 的大小无关, 共 8 次赋值。整段代码进行 $O(1) + 8(n - 4) = O(n)$ 次赋值。

尽管存在嵌套循环, 算法的整体时间复杂度依然呈线性增长。
看的是「内层跑几次」, 不是「有几层循环」。

对于排序算法, 主要时间开销体现在比较和交换(移动)上, 见第 8 章。

1.4.2 最佳、最差和平均情况

即使问题规模相同, 输入数据不同, 时间复杂度也可能不同——因为算法实际执行的操作往往依赖于分支条件的走向, 而分支走向又取决于输入数据的取值。

原书举的例子是「求一个数组的所有有序子数组中最长的一个」。
在 $[1, 8, 1, 2, 5, 0, 11, 9]$ 中, 最长的有序子数组是 $[1, 2, 5]$, 长度为 3。

```

1  for (i = 0, length = 1; i < n - 1; i++) {
2      for (j1 = j2 = k = i; k < n - 1 && a[k] < a[k + 1]; k++, j2++);
3      if (length < j2 - j1 - 1)
4          length = j2 - j1 + 1;
5  }

```

⚠ 上面这段是原书印出来的样子, 照抄它会算错。

内层循环走完这一段升序的长度是 $j2 - j1 + 1$, 而判断条件写的却是 $length < j2 - j1 - 1$, 两处的常数项差了 2。用原书自己的例子跑一遍, 答案会是 2 而不是 3。这里保留原样是为了让读者对照纸书, 分析长度时按 $j2 - j1 + 1$ 读(见 [book/勘误.md](#))。

按输入的初始状态分情况:

1. 全部降序输入: 外层执行 $n - 1$ 次, 每次内层只执行 1 次, $O(n)$;
2. 全部升序输入: 外层执行 $n - 1$ 次, 对每个 i 内层执行 $n - 1 - i$ 次, $O(n^2)$;
3. 平均情况: 元素无序, 介于两者之间。

计算平均情况的复杂度需要考虑所有输入情况。每种输入等概率时, 把各自的操作数相加再除以输入总数即可; 概率不等时要把概率当权值:

$$C_{\text{avg}} = \sum_i p(\text{input}_i) \text{steps}(\text{input}_i)$$

其中 $p(\text{input}_i) \geq 0$ 且 $\sum_i p(\text{input}_i) = 1$ 。

但平均情况的分析并不总是可行——它要求你事先知道输入的概率分布。

以在规模为 n 的数组中顺序检索一个给定的 K (假设有且仅有一个元素等于 K) 为例:

情况	K 在哪儿	比较次数
最佳	第 1 个元素	1
最差	最后一个元素	n
平均 (每个位置等概率 $1/n$)	任意位置	$\frac{1+2+\cdots+n}{n} = \frac{n+1}{2}$
平均 (第 1 位 $1/2$ 、第 2 位 $1/4$ 、其余均分 $\frac{1}{4(n-2)}$)	偏向前面	$\frac{1}{2} + \frac{2}{4} + \frac{3+\cdots+n}{4(n-2)} = 1 + \frac{n+3}{8}$

最后一行比等概率的 $\frac{n+1}{2}$ 快将近 4 倍——概率分布变了，结论就明显改变。

那该研究哪一种情况？

- **最佳情况**发生的概率太小，不能作为算法性能的代表；但它能帮助设计者或使用者了解某个算法在何种情况下适用。
- **最差情况**让人了解一个算法至少能做多快，这一点在实时系统中尤其重要：在空运处理系统中，一个绝大部分情况下能管理 n 架飞机、但不能在规定时间内做到的算法，是不可接受的。
- 对多数算法而言，最坏情况和平均情况的公式往往只差常数因子或常数项。

算法分析的第一条纪律：只报一个数字、不说针对哪种输入，这个数字就没有意义。

本书后面每讲一个算法，都会分别交代它的最佳、最差与平均情况——

例如第 8 章的快速排序平均是 $O(n \log n)$ ，而输入已经有序时退化成 $O(n^2)$ 。

1.4.3 时间和空间的折衷

空间开销也可以用渐进分析。但很多常见算法使用的是**静态存储结构**——

所谓静态，是指算法所使用的存储空间在执行过程中并不发生变化，

一旦确定了输入数据和问题规模，数据结构大小也就确定了。

这类结构的时空开销往往与问题规模成正比（线性）或不随规模增大（常数），

比较容易估算，本书对这类算法一般只讨论时间代价。

也经常会遇到使用**动态数据结构**的算法，其存储空间在运行过程中会有数量级的增大或缩小，这时空间开销的分析是十分必要的。

时空资源的折衷原理：对于同一个问题，多种算法在时间和空间开销上的优劣

往往表现出「时空折衷」的性质——为了改善时间开销，往往要以增大空间开销为代价；反之亦然。

- **空间换时间**。为了从公司黄页数据表中快速查询电话，可以附加一个散列式索引表：用散列函数的简单计算代替在黄页中逐项比较公司名称，时间开销从线性改善为常数，空间开销则增添一个线性增长率的存储空间。
- **时间换空间**。第 6.3 节把一棵树压成一串「带右链」「带度数」的序列，省下了全部指针域的空间，代价是恢复结构时必须重新扫描一遍；第 12 章的稀疏矩阵、Huffman 编码也都是同一个取舍的不同侧面。

没有免费的加速：空间紧、时间松，和空间松、时间紧，选的结构会不一样。

1.4.4 求解问题时数据结构的选择和评价

利用计算机求解一个给定问题时，一个关键任务是确立问题模型所采用的数据结构。

一旦确定了数据结构，算法的设计便水到渠成。可以遵循以下原则：

1. 仔细分析所要解决的问题，特别是求解问题所涉及的数据类型和数据间的逻辑关系；
2. 在算法设计之前往往先进行数据结构的初步设计；

3. 注意数据结构的**可扩展性**——输入规模改变时结构是否适应，能否适应问题的演变和扩展；
4. 比较算法**时空开销**的优劣。

落到本章的例子：1.1 节要的是任意两点之间的最短路径，并且只有五个经纪人，所以相邻矩阵加 Floyd 是合适的——实现简单，三重循环在 $n = 5$ 时可以忽略。

- 如果顶点很多、边很少，而且只问单源最短路径，就应该换成邻接表加 Dijkstra（第 7 章）；
- 再往下还要看数据在内存还是外存：一次磁盘 I/O 往往比一次比较贵几个数量级，这是第 9 章与第 11 章反复出现的分界线。

本章小结

- 利用计算机求解问题的过程，是一个问题抽象、数据抽象和算法抽象的过程。
分析需求 → 抽象出模型 → 确定数据结构、设计算法 → 编码实现。
- 数据结构 = 逻辑结构 + 存储结构 + 运算。
 - 逻辑结构是从具体问题抽象出来的数学模型 (K, r) ，按关系 r 的性质分为线性结构、树形结构和图结构三大类；
 - 存储结构是从逻辑结构到物理存储的一种映射，基本方法有顺序、链接、索引、散列四种；同一逻辑结构可以有多种存储表示。
- 抽象数据类型 ADT 是定义了一组操作的抽象模型，帮助人们独立于实现细节理解数据结构的性质与约束，达到数据抽象、信息隐蔽。
- 算法是对特定问题求解过程的描述，是指令的有限序列，应具备通用性、有效性、确定性、有穷性。
设计方法有穷举、回溯、分治、递归、贪心、动态规划等。
- 算法分析不看墙上的秒数，而看基本操作次数随规模 n 怎么长，用 O 、 Ω 、 Θ 说话，并分清最好、最坏和平均。
- 同一问题的多种算法往往表现出时空折衷，需要根据应用具体情况权衡。

习题与参考答案

本节是选讲，不是全集。这里收的是补充题、正文习题与上机题中挑出来讲的一部分；原书全部 345 道习题与上机题的参考答案，在 [../book/习题与参考答案.md](#)。那份文件由 `tools/check_doc.py` 的 R14 逐题守着——每一道正文题都必须有同号答案，否则闸门变红。（闸门只验「有没有同号答案」，不验答案内容是否正确。）

补充证明练习

1. 精确计算下列双重循环中赋值语句 `a[i][j] = 1` 的执行次数，再给出数量级：

```
1  for (i = 1; i < n; ++i)
2      for (j = n; j >= i; --j)
3          a[i][j] = 1;
```

答：外层 $i = 1 \dots n-1$ ，内层执行 $n-i+1$ 次，
因此赋值次数为 $\sum_{i=1}^{n-1} (n-i+1) = \frac{n(n+1)}{2} - 1$ ，为 $\Theta(n^2)$ 。

2. 对非负渐进函数证明 $\max(f(n), g(n)) = \Theta(f(n) + g(n))$ 。

答：由 $\max(f, g) \leq f + g \leq 2 \max(f, g)$ ，取 $c_1 = 1/2$ 、 $c_2 = 1$ 即得 Θ 关系。

3. 求解并用数学归纳法证明 $T(n) = 2T(\lfloor n/2 \rfloor) + n$ 的渐进阶。

答：递归树每层总成本都是 n ，深度为 $\Theta(\log n)$ ，故 $T(n) = \Theta(n \log n)$ ；
归纳证明用代入法分别验证上、下界即可。

正文习题

1. 设计一个算法，从大到小依次输出顺序读入的三个整数 X 、 Y 、 Z 。

答：读入后三次比较即可定序：若 $X < Y$ 交换，若 $X < Z$ 交换，若 $Y < Z$ 交换，此时 $X \geq Y \geq Z$ ，顺次输出。时间 $O(1)$ 、空间 $O(1)$ 。

注意「从大到小」允许相等，判断要用 $<$ 而不是 $<=$ ，否则相等元素会白白交换。

2. 举例说明什么是数据结构。

答：数据结构 = 一组数据 + 它们之间的关系 + 允许的运算。

「班级花名册」按学号排成一列、前后相邻，是线性表；

「城市间的航线」两两之间可能有边，是图；「文件夹套文件夹」是树。

同一批数据换一种关系去看，就是另一种数据结构，能做的运算和代价也随之变化。

3. 数据有哪些存储方式？各有什么特点。

答：顺序（元素连续存放，靠下标算地址，随机访问 $O(1)$ ，插删要移动）、

链接（结点分散、靠指针相连，插删 $O(1)$ 但不能随机访问，每个结点多花指针空间）、

索引（另建索引表指向记录，逻辑有序而物理无序，见第 11 章）、

散列（由关键码直接算地址，平均 $O(1)$ ，需处理冲突，见第 10 章）。

4. 分析下面程序段中循环体执行多少次。

```
1  int i = 0, s = 0, n = 100;
2  do {
3      i = i + 1;
4      s = s + 10 * i;
5  } while ((i < n) && (s < n));
```

答：循环体执行 4 次。do-while 先执行后判断：

$i = 1$ 时 $s = 10$ ， $i = 2$ 时 $s = 30$ ， $i = 3$ 时 $s = 60$ ， $i = 4$ 时 $s = 100$ ，

此时 $s < n$ 不再成立，退出。

一般地 $s = 5i(i+1)$ ，循环次数由 $5i(i+1) \geq n$ 决定，是 $O(\sqrt{n})$ 而不是 $O(n)$ ——

这道题要的就是别被 $i < n$ 那一半条件骗过去。

5. 指出算法 A 的功能和时间复杂度。 h 、 g 是同一个单循环链表上的两个结点指针。

```
1  void B(Node* s, Node* q) {
2      Node* p = s;
3      while (p->next != q) p = p->next;
4      p->next = s;
5  }
6  void A(Node* h, Node* g) {
7      B(h, g);
8      B(g, h);
9  }
```

答：B(s , q) 从 s 出发走到「下一个是 q 」的结点 p ，令 $p->next = s$ ，于是 s 到 p 这一段自成一个循环链表。

A(h , g) 先后做 B(h , g)、B(g , h)，把一个循环链表在 h 与 g 两处剪开，拆成两个循环链表。两次遍历各走一段，合起来正好绕原表一圈，

时间 $\Theta(n)$ 、空间 $\Theta(1)$ 。

6. 设 n 是偶数，计算下列程序段结束后 m 的值，并给出时间复杂度。

```
1  m = 0;
2  for (i = 1; i <= n; i++)
3      for (j = 2 * i; j <= n; j++)
4          m = m + 1;
```

答：内层对每个 i 执行 $\max(0, n - 2i + 1)$ 次，只有 $i \leq n/2$ 才进入，于是
$$m = \sum_{i=1}^{n/2} (n - 2i + 1) = \frac{n}{2}(n + 1) - 2 \cdot \frac{(n/2)(n/2 + 1)}{2} = \frac{n^2}{4}.$$

($n = 10$ 时 $m = 25$, $n = 100$ 时 $m = 2500$ 。) 时间 $\Theta(n^2)$ 。

7. 把下列运行时间写成大 O : $T_1(n) = 1000$, $T_2(n) = n^2 + 1000n$, $T_3(n) = 3n^3 + 100n^2 + n + 1$ 。

答： $O(1)$ 、 $O(n^2)$ 、 $O(n^3)$ 。大 O 只保留增长最快的项并丢掉常数因子——
1000 再大也是常数。

8. 给出下面两个算法的时间复杂度：(1) 一层循环 $i = 1..n$ 做 $x = x + 1$ ；
(2) 两层循环 $i, j = 1..n$ 做 $x = x + 1$ 。

答：(1) $O(n)$ ；(2) $O(n^2)$ 。

9. 给出求两个整数最大公因数的算法，并分析时间和空间复杂度。

答：辗转相除 $\gcd(a, b) = \gcd(b, a \bmod b)$, $b = 0$ 时答案是 a 。
最坏情形是相邻的斐波那契数，迭代次数 $O(\log \min(a, b))$ ；
迭代写法空间 $O(1)$ ，递归写法额外占 $O(\log \min(a, b))$ 的调用栈。

10. 计算机每秒 10^{10} 次操作，要求一小时内算完。下列复杂度各能处理多大的 n ：
 n^2 , n^3 , $100n^2$, $n \log_{10} n$, 2^n , 2^{2^n} 。

答：一小时的预算是 $10^{10} \times 3600 = 3.6 \times 10^{13}$ 次操作。

复杂度	可处理的 n
n^2	6.0×10^6
n^3	3.3×10^4
$100n^2$	6.0×10^5
$n \log_{10} n$	$\approx 2.9 \times 10^{12}$
2^n	45
2^{2^n}	5

最后两行是这道题的重点：机器再快一倍， 2^n 也只多算一个 n ，
 2^{2^n} 连一个都不一定多得了。

11. 已知 $f(n)$ 是 $O(g(n))$ 。判断并证明或举反例：

(1) $\log_2 f(n)$ 是 $O(\log_2 g(n))$ ；(2) $2^{f(n)}$ 是 $O(2^{g(n)})$ ；(3) $f(n)^2$ 是 $O(g(n)^2)$ 。

答：

(1) 不成立。取 $f(n) = 2$ 、 $g(n) = 1$ ，则 $f = O(g)$ ，但 $\log_2 f = 1$ 、 $\log_2 g = 0$ ，
 $1 = O(0)$ 不成立。（若另加 $f, g \geq 2$ 的前提则成立。）

(2) 不成立。取 $f(n) = 2n$ 、 $g(n) = n$ ， $2^{2n} = (2^n)^2$ 不是 $O(2^n)$ 。

(3) 成立：由 n 充分大时 $f \leq cg$ 得 $f^2 \leq c^2 g^2$ 。

12. 计算下列递推的复杂度: (1) $T(n) = T(\lceil n/2 \rceil) + 1$;
(2) $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$; (3) $T(n) = 2T(\lfloor n/2 \rfloor) + n$ 。

答: (1) $\Theta(\log n)$ ——每次规模减半、只加常数;
(2) $\Theta(n)$ ——递归树有 n 个叶、每个结点只花常数, 总和与结点数同阶;
(3) $\Theta(n \log n)$ ——每层总代价都是 n , 共 $\Theta(\log n)$ 层。

上机题与参考实现

上机题 1 分拆计数

m, n 是自然数。 m 可以写成若干个不超过 n 的自然数之和,
编写 $f(m, n)$ 计算有多少种写法。例如 $f(5, 3) = 5$:
 $3 + 2, 3 + 1 + 1, 2 + 2 + 1, 2 + 1 + 1 + 1, 1 + 1 + 1 + 1 + 1$ 。

思路。按「这一堆里用不用一个 n 」分成互不重叠的两类:

$$f(m, n) = \underbrace{f(m-n, n)}_{\text{用掉一个 } n} + \underbrace{f(m, n-1)}_{\text{一个 } n \text{ 都不用}}$$

边界: $f(0, n) = 1$ (空拆法算一种); $m > 0$ 且 $n = 0$ 时为 0; $n > m$ 时按 $f(m, m)$ 算。

直接照抄递推式写递归会重复计算同一个 (m, n) 无数次, 改成二维递推即可,
时间 $O(mn)$ 、空间 $O(mn)$ (可滚动到 $O(m)$)。这就是第 12 章动态规划的雏形。

完整程序见 [code/ch01/partition_count.cpp](#), 核心是:

```
1 [[nodiscard]] long long partition_count(int m, int n) {
2     if (m < 0 || n < 0) {
3         return 0;
4     }
5     std::vector<std::vector<long long>>> f(
6         static_cast<std::size_t>(m) + 1,
7         std::vector<long long>(static_cast<std::size_t>(n) + 1, 0));
8
9     for (int part = 0; part <= n; ++part) {
10         f[0][static_cast<std::size_t>(part)] = 1;    // 和为 0: 只有「什么都不取」一种
11     }
12     for (int total = 1; total <= m; ++total) {
13         for (int part = 1; part <= n; ++part) {
14             const auto t = static_cast<std::size_t>(total);
15             const auto p = static_cast<std::size_t>(part);
16             f[t][p] = f[t][p - 1];                    // 一个 part 都不用
17             if (total >= part) {
18                 f[t][p] += f[t - static_cast<std::size_t>(part)][p];    // 用掉一个 part
19             }
20         }
21     }
22     return f[static_cast<std::size_t>(m)][static_cast<std::size_t>(n)];
23 }
```

```
1 c++ -std=c++17 -Wall -Wextra -Werror courseware/code/ch01/partition_count.cpp -o /tmp/partition
2 /tmp/partition
```

```

1  f(5, 3)    = 5
2  f(10, 10) = 42
3  f(50, 50) = 204226

```

程序里带了断言自检： $f(5, 3) = 5$ 是题面给的例子；
 $n \geq m$ 时就是 m 的分拆数 $p(m)$ (1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42, ...) ,
 所以 $f(10, 10) = 42$ ；每份只能是 1 时无论 m 多大都只有一种写法；
 一个数都不许用时 $m > 0$ 无解。

上机题 2 复数 ADT

试用 C++ 的类声明定义「复数」的抽象数据类型。要求：

- ① 内部用浮点数定义实部和虚部；② 实现 3 个构造函数（无参 / 一个双精度数 / 两个双精度数）；③ 定义获取和修改实部虚部，以及 `+` `-` `*` `/` 的成员函数；
- ④ 定义重载的流函数来输出一个复数。

这道题的要点全在接口上，不在算术上。完整程序见

[code/ch01/complex_adt.cpp](#)，五条设计决定：

1. 三个构造函数不要用默认参数塞成一个 —— 题目要的就是重载。
2. 单参数构造要写 `explicit`，否则 `Complex c = 3.0;` 这种隐式转换会悄悄发生。
3. 四则运算写成自由函数（或 `friend`）而不是成员函数 —— 成员版的左操作数必须是 `Complex`，写不出 `2.0 + c` 这种对称接口。
4. 除法先判分母模长为零并抛 `std::invalid_argument` —— 按 D-001：ADT 内部不打印任何东西，真错误靠异常。
5. `operator<<` 只负责格式化，不做任何判断。

```

1  class Complex {
2  public:
3      // 题目要的是三个**重载**的构造函数，不是一个带默认参数的。
4      Complex() : real_(0.0), imag_(0.0) {}
5      // explicit: 否则 `Complex c = 3.0;` 这种隐式转换会悄悄发生。
6      explicit Complex(double real) : real_(real), imag_(0.0) {}
7      Complex(double real, double imag) : real_(real), imag_(imag) {}
8
9      [[nodiscard]] double real() const { return real_; }
10     [[nodiscard]] double imag() const { return imag_; }
11     void set_real(double value) { real_ = value; }
12     void set_imag(double value) { imag_ = value; }
13
14     [[nodiscard]] double norm() const { return real_ * real_ + imag_ * imag_; }
15
16 private:
17     double real_;
18     double imag_;
19 };
20
21 inline Complex operator/(const Complex& a, const Complex& b) {
22     const double d = b.norm();
23     if (d == 0.0) {
24         throw std::invalid_argument("complex division by zero");
25     }
26     return {(a.real() * b.real() + a.imag() * b.imag()) / d,
27             (a.imag() * b.real() - a.real() * b.imag()) / d};
28 }

```

```
1 | c++ -std=c++17 -Wall -Wextra -Werror courseware/code/ch01/complex_adt.cpp -o /tmp/complex-adt
2 | /tmp/complex-adt
```

```
1 | a      = 1 + 2i
2 | b      = 3 - 4i
3 | a * b   = 11 + 2i
4 | a / b   = -0.2 + 0.4i
5 | a / 0   -> 抛出 std::invalid_argument
```

不要用 `std::complex` 代替——这道题练的正是「自己定义一个类型」。

上机题 3 把 1.1 节的程序改一改

不必新写代码，直接改 [../code/ch01/adt/demo.cpp](#) 再编译运行：

1. 删掉 `network.add_route(2, 1, 7);`，观察 `best_source()` 返回空值时程序的行为；
2. 把 `B3 -> B4` 的时间从 10 改成 1，确认答案仍是 B3、最慢传播时间变成 7；
3. 把 Floyd 的最外层循环 `via` 挪到最内层，用图 1.3 的结果矩阵对照，看看哪些格子算错了——这是理解「为什么最外层必须是中转点」最快的办法。